# 1. Einführung in das Betriebssystem Linux

## 1.1. Kurzer geschichtlicher Abriss

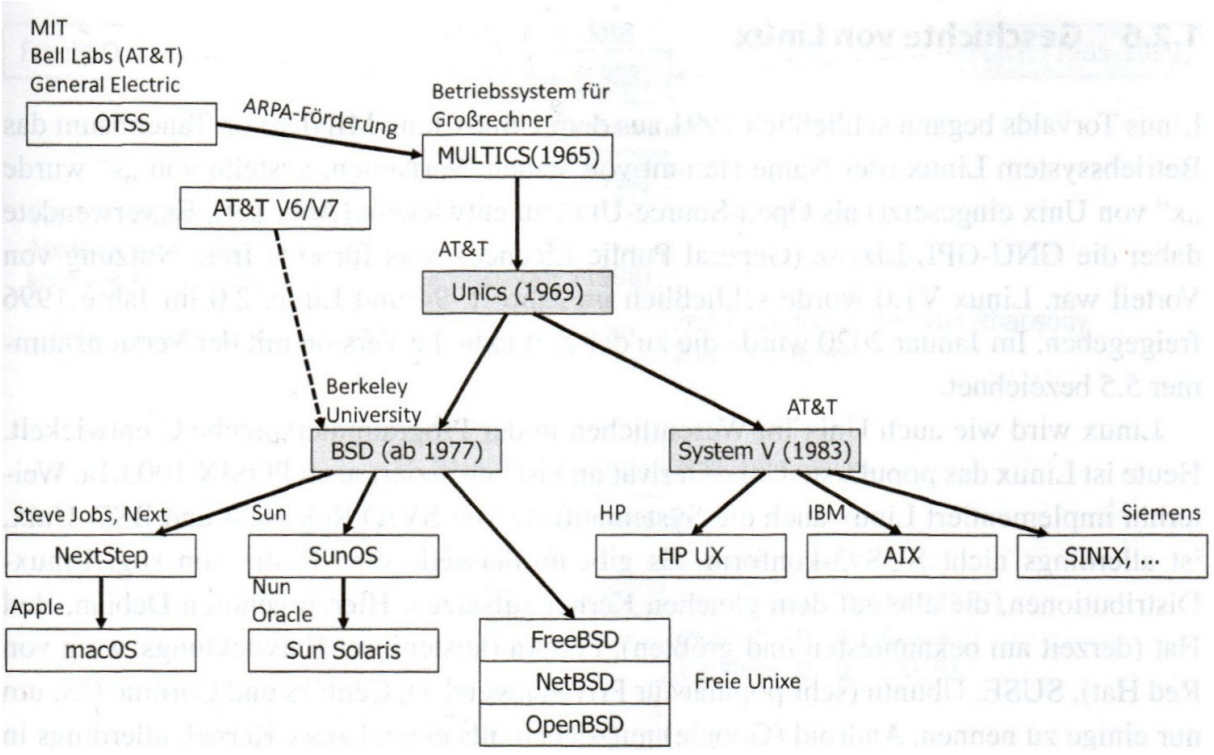
Linus Torvalds begann 1991 mit der Entwicklung von LINUX. Grudlage war das MINIX-Clone (Unix-Derivat) von Tannenbaum.

Name: Vorname des Entwicklers, aber das „s“ durch ein „x“ ersetzt, das an Unix erinnern soll.

Thorvalds verwendete bei der Veröffentlichung (erste Veröffentlichung 1994) die GNU-GPL-Lizenz (GPL = General Public Licence), was für eine freie Nutzung von Vorteil war.

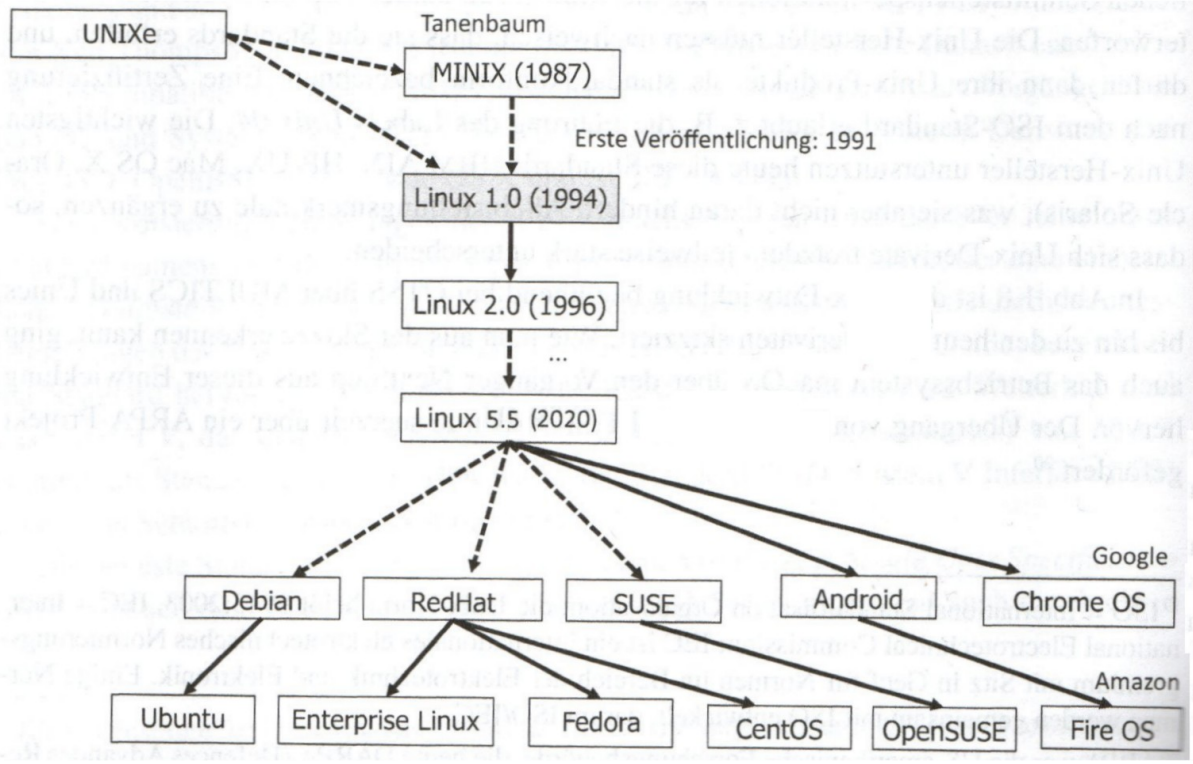
Exkurs: Blick in die Geschichte von Unix-Systemen:

Unix entstand aus dem Betriebssystem MULTICS und wurde unter einer PDP-7 von Ken Thomsam und Dennis Ritchie in den Bell Labs entwickelt. Danach entstanden zwei inkompatible Varianten: BSD-Unix (Berkely University) und System V (Unix System Laboratories von AT&T).




Zurück zu Linux: Linux wird wie Unix in der Programmiersprache C entwickelt und ist heute das populärste Unix-Derivat. Es ist zertifiziert nach POSIX 1003.1a. Weiterhin realisiert Linux auch die Systemaufrufe von SVID Rel. 4 und BSD-Unix.

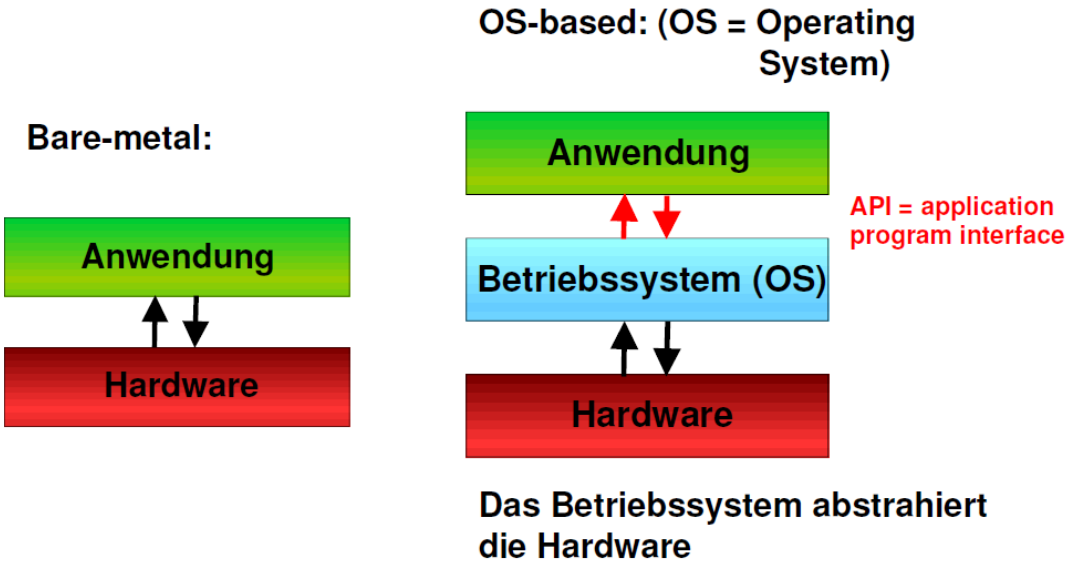
Es gibt mittlerweile eine Reiher von sogenannte Linux-Distrubutionen, die alle auf dem gleichen Kernel aufsetzen.




## 1.2. Eigenschaften des Linux-Kernels

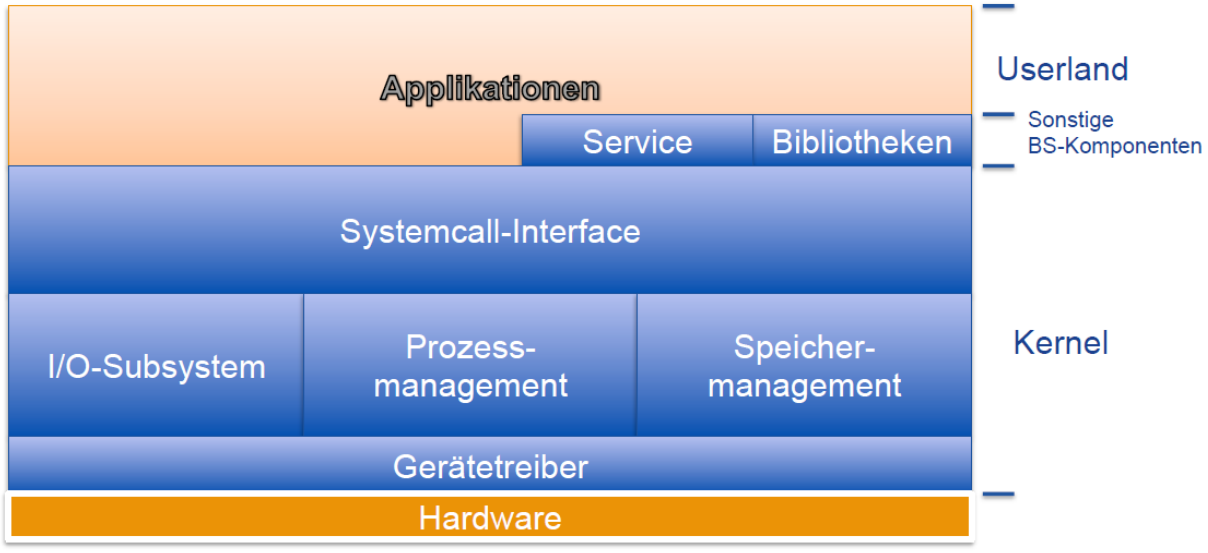
Um die speziellen Eigenschaften des Linux-Kernels beschreiben zu können, soll zunächst auf die allgemeinen Aufgaben eines Betriebssystems eingegangen werden.

Die primäre Aufgabe eines Betriebssystems ist die Abstraktion der Hardware: der Programmierer einer Anwendung braucht keine Hardware-Kenntnisse mehr, dafür muss der das Appliction Program Interface (API) kennen oder über Bibliotheksfunktionen aufrufen.




Eingebettete Betriebssysteme: Bare-metal vs. OS-based

Den grundsätzlichen – etwas detaillierteren - Aufbau (Architektur) eines Betriebssystems zeigt die folgende Darstellung:




Zu dem Betriebssystem zählt man nicht nur den Betriebssystem-Kern (Kernel), sondern auch Bibliotheken und Dienste (Services; ausgeführt von Hintergrundprozessen, engl. daemon), die im Usermodus (siehe unten) ablaufen. Das Systemcall-Interface implementiert die API-Funktionen (auch Systemcalls genannt) und läuft wie die restlichen Komponenten im Kernel-Modus. Durch die Verwendung der Systemcalls greift eine Applikation auf die Hardware

zu (ist z. Bsp. in der Lage, Daten auf dem Bildschirm darzustellen oder Daten aus einer Datei auszulesen).

Zunächst einmal soll die Unterscheidung zwischen Usermodus (Benutzermodus) und Kernelmodus erläutert werden: Im Kernelmodus läuft die Software **privilegiert** ab, im Bentzermodus **nicht privilegiert**. Das hat bestimmte Einschränkungen zur Folge:

- Es sind bestimmte Prozessorbefehle nicht verfügbar (bestimmtes Bit in einem Steuerregister des Prozessors ist entsprechend gesetzt).
- Es ist nur ein Teil des Adressraums verfügbar (Seitentabellen der MMU=Memory Management Unit sind entsprechend gebaut).

## Prozessverwaltung:

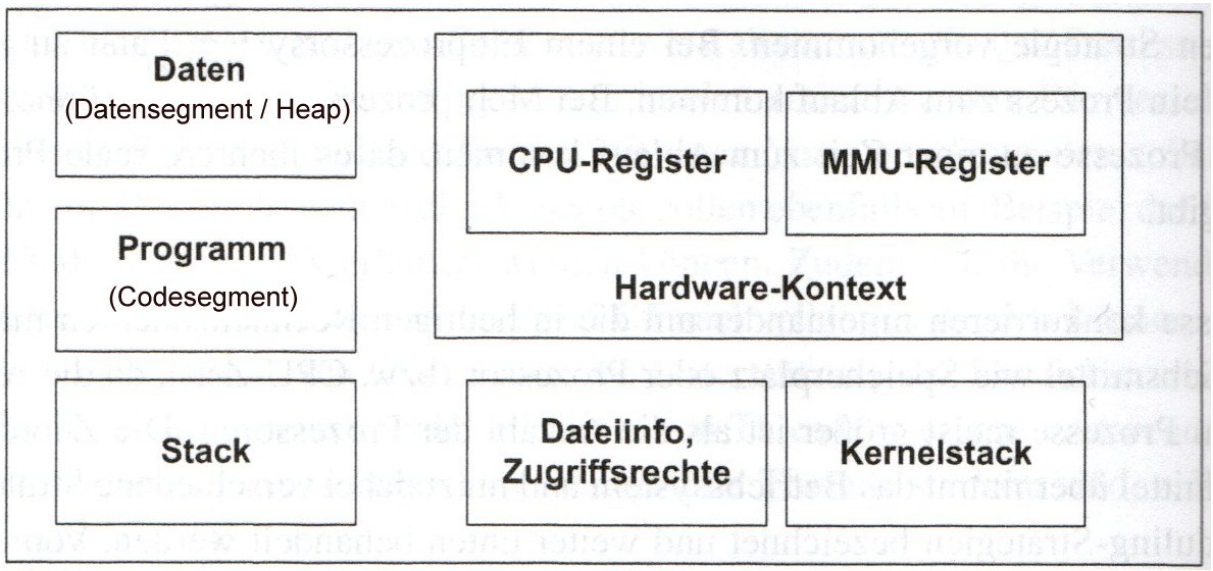
Das Prozess- und Threadmodell sind wesentliche Konzepte des Betriebssystems und dienen als grundlegende Bausteine der Parallelverarbeitung (Multiprocessing).

Ein Prozess (manchmal auch Task genannt) ist die Ablaufumgebung eines Programmes, die vom Betriebssystem zur Verfügung gestellt wird. Das Betriebssystem stellt dem Programm einen Adressraum zur Verfügung, wohin das ausführbare Programm geladen wird. Ein Prozess ist damit ein Programm zur Laufzeit bzw. die konkrete Instanzierung eines Programms innerhalb eines Rechensystems.

Prozesse konkurrieren miteinander um die „Betriebsmittel“ wie Speicherplatz oder Prozessor. Die Anzahl der laufenden Prozesse ist mest größer als die Anzahl der Prozessoren.

Die Zuordnung der Betriebsmittel übernimmt das Betriebssystem und nutzt dabei verschiedene Strategien (**Scheduling**-Strategien).

Das Betriebssystem verwaltet für jeden Prozess vielfältige Informationen, die als **Prozess-Kontext** bezeichnet werden. Dazu zählt man Programme und Daten des Prozesses und die Inhalte aller Hardware-Register (PC, PWS, MMU-Register, sonstige Register). Die Registerinhalte werden auch als Hardware-Kontext bezeichnet.




Verliert der gerade aktive Prozess die CPU, so muss ein Kontextwechsel durchgeführt werden (Kontext-Switching). Dabei wird der Hardware-Kontext des laufenden Prozesses gesichert und der Hardwarekontext des neu aktivierten Prozesses in die Ablaufumgebung geladen.

Der Adressraum eines Prozesses wird meist vom Compiler in verschiedene Segmente aufgeteilt:

• Codesegment für das ausführbare Programm

• Stacksegment (Stapelspeicher) für lokale Variabeln und Rücksprungadressen

• Datensegment für statische (globale) Variablen. Dynamisch allokierte Variablen werden im sogenannten Heap (Halde) abgelegt.

Demo: **`ap0/pagesize.c`**

Wenn ein Prozess Code des Kernels ausführt (Systemcall), werden werden die für den Prozess relevanten Rücksprungadressen in einem Kernelstack abgelegt. Die MMU ist zuständig für die Berechnung der physikalischen Adressen, die prozess-spezifischen Register gehören mit zum Hardware-Kontext.

## Process Controll Block:

Neben Tabellen zur Speicherverwaltung und zur Dateiverwaltung führt das Betriebssystem auch eine Tabelle der aktuellen Prozesse.

Jedem Prozess ist ein Eintrag in dieser Liste zugeordnet, der auch als Prozesskontrollblock (PCB) oder Taskkontrolblock (TCB) bezeichnet wird.

Der PCB enthält unter anderem die Information über zugeordnetes Code-, Daten- und Stacksegment und die Information, wo der gesamte Prozesskontext abgespeichert ist. Auch **Prozesszustand** (siehe weiter unten), Prozesspriorität (initial und dynamisch) und Prozessidentifikationsnummer (**PID = process identification**) sind verfügbar. Auch die Information über den erzeugenden Prozess (**PPID** = parent process identification) ist verfügbar.

Der erzeugende Prozess wird auch Vaterprozess oder Elternprozess genannt. In einem Unix-System (damit auch in Linux) hat der vom Kernel nach dem Bootvorgang gestartete Prozess die PID 1 (der `Init` -Prozess). Alle weiteren Prozesse werden – direkt und indirekt – als Kindprozess des `Init` -Prozesses erzeugt. Letztendlich sind also alle Prozesse Kindprozesse von `Init`. Startet man selbst einen Prozess aus einer Shell heraus, ist der Prozess immer auch ein Kindprozess der laufenden Shell.

Demo: **`ap0/pid.c`**

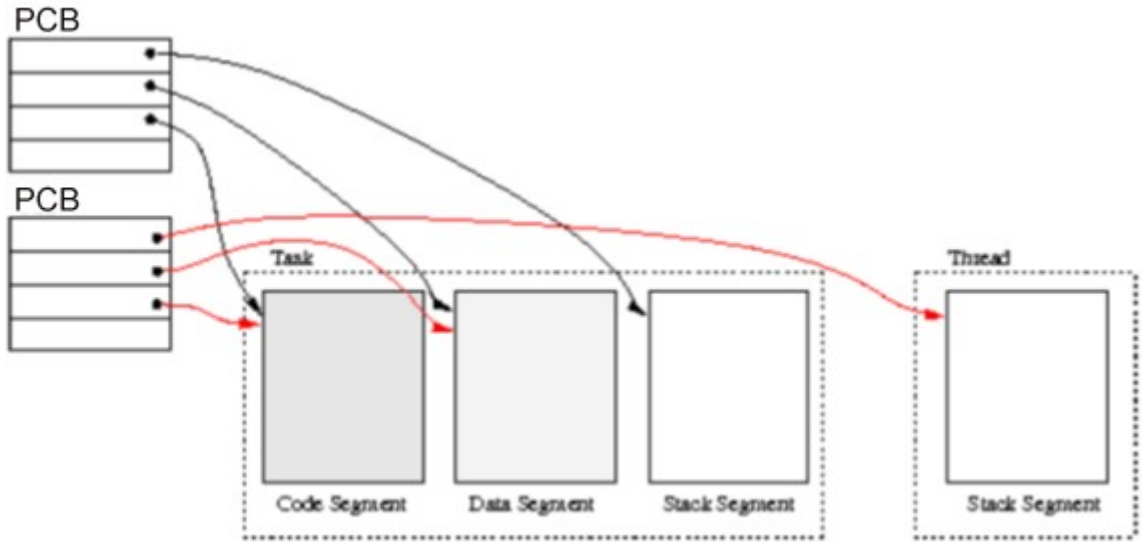
Leichtgewichtige Prozesse (Threads):

Die Verwaltung der Betriebsmittel eines Prozesses ist relativ aufwändig. Ein ergänzendes, ressourcenschonendes Konzept sind die sogenannten Threads (Fäden), die auch als leichtgewichtige Prozesse bezeichnet werden (LWP = light-weight process). Alle Threads eines Prozesses teilen sich den gemeinsamen Adressraum (Code- und Datensegment). Sie haben auch Zugriff auf offenen Dateien und Netzwerkverbindungen des Prozesses.

Allerdings haben sie ein eigenes Stacksegment, um lokale Variablen dort abzulegen. Sie verfügen auch über einen eigenen Threadkontext und konkurrieren untereinander und mit den weitern Prozessen um die Zuteilung einer CPU.

Die folgende Abbildung skizziert den Prozess-Control-Block (PCB) für zwei Prozesse; bei dem zweiten (rote Pfeile) handelt es sich um einen weiteren Ausführungsfaden (=Thread) des ersten Tasks. Das ist

daran zu erkennen, dass das Codesegment und das Datensegment gemeinsam genutzt wird.

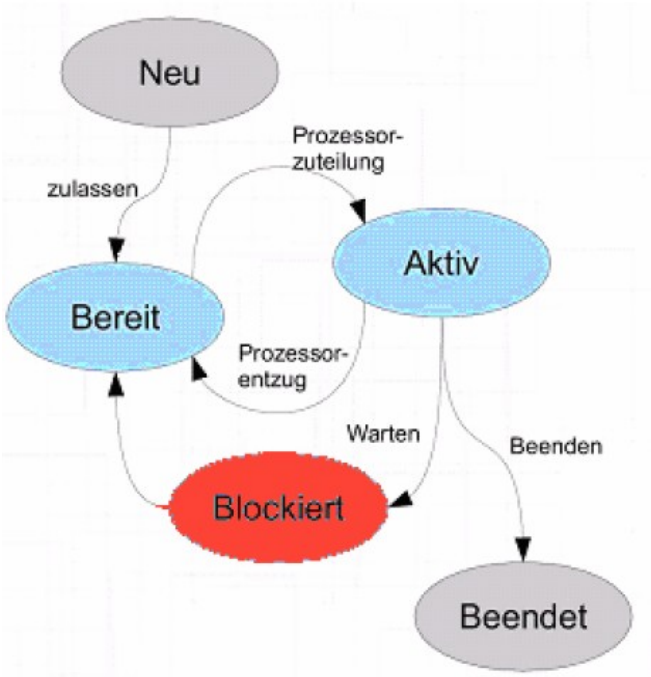



Demo: **`ap0/fork1.c` und `ap0/fork2.c`**

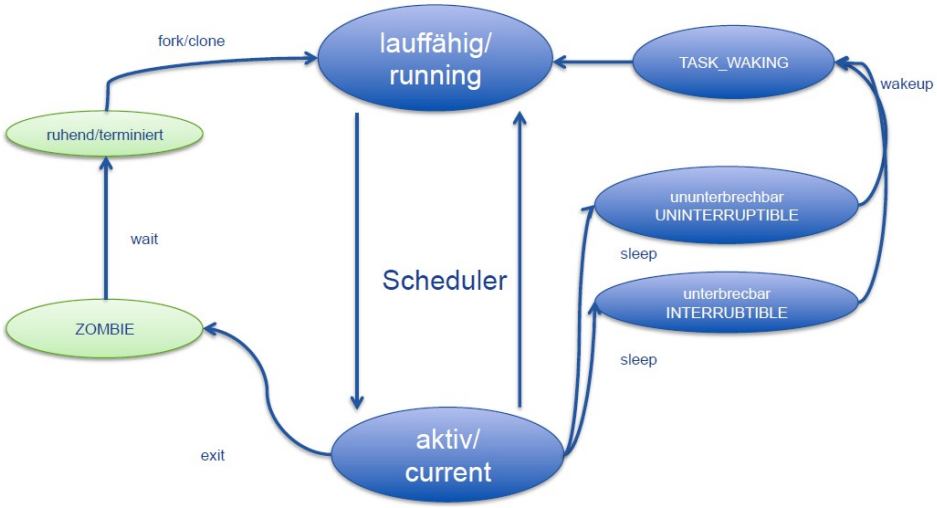
Prozesslebenszyklus:

Ein Prozess wird durch einen Systemcall erzeugt (z. Bsp. `fork` bei Linux). Dabei werden der Programmcode und die Daten in den Speicher geladen. Innerhalb des Betriebssystems wird dem Prozess eine eindeutige Identifikation zugeordnet.

Ein Prozess kann normal bei Programmende beendet werden, aber es besteht auch die Möglichkeit, dass das Betriebssystem den Prozess beendet. Während der Lebensdauer durchläuft ein Prozess verschiedene Zustände: Die wichtigsten drei werden meist mit „Aktiv“, „Bereit“ und „Blockiert“ bezeichnet.




In Linux gibt es noch mehr Prozesszustände: z. Bsp. wird beim Schlafen (anderer Namen für Blockiert) wird zwischen unterbrechbar und nicht-unterbrechbar unterschieden:



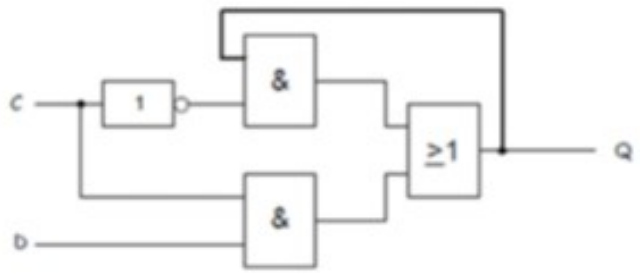

Neben dem **Scheduling**, dem Erstellen und Zerstören von Prozessen, gehört es auch zu den Aufgaben des Betriebssystems, Mechanismen zur zwischen einzelnen Prozessen zur Verfügung zu stellen (IPC = Inter Process Communication).

## Nebenläufigkeit von Prozessen:

Mehrere Prozesse laufen quasi gleichzeitig ab (in dem Sinn, dass Sie gleichzeitig aktiv sind und von Zeit zu Zeit die CPU entzogen wird). Man spricht in diesem Zusammenhang von der Nebenläufigkeit der Prozesse. Das gleiche Programmstück kann auch in verschiedenem Prozesszusammenhang (quasi gleichzeitig) betreten werden. Das Programmstück muss „Reentry“-fähig sein.

Das grundsätzliche Problem tritt aber schon auf, wenn zwei Prozesse auf gleiche Ressourcen (z. Bsp. Speicherbereiche) zugreifen: Es kann zu so genannten Race-Conditions (Wettlaufsituationen) kommen, wo dann das Ergbenis nicht mehr determiniert ist, sondern vom Zufall abhängt (wann nämlich ein Prozess unterbrochen wird und dann vom anderen überholt werden kann).

Race-Conditions sind aus der digitalen Hardware bekannt: Die korrekte Funktion ist auf Grund von Wettrennsituationen abhängig von der Laufzeit der Gatter. Ein Beispiel heirfür ist die folgende Realisiserung eines taktzustandsgesteuerten D-Flip-Flops:




| C | D | Q | Q+ | |
|---|---|---|---|---|
| 0 | 0 | 0 | 0 | Speichern |
| 0 | 0 | 1 | 1 | Speichern |
| 0 | 1 | 0 | 0 | Speichern |
| 0 | 1 | 1 | 1 | Speichern |
| 1 | 0 | 0 | 0 | Rücksetzen |
| 1 | 0 | 1 | 0 | Rücksetzen |
| 1 | 1 | 0 | 1 | Setzen |
| 1 | 1 | 1 | 1 | Setzen |

Fehlfunktionen sind bei Parallelausführung (Nebenläufigkeit) von Codeabschnitten auch in der Software denkbar. Auch dies lässt sich an einem denkbar einfachen Beispiel aufzeigen:

Neue Denkweise: Der Programmcode kann „gleichzeitig“ von mehreren Prozessen genutzt werden

Bsp. für ein einfaches Programm:

(1) Wert aus Speicher in Register lesen

(2) Register inkrementieren

(3) Wert aus Register in Speicher zurück schreiben

Wenn der Wert im Speicher zu Beginn 1 ist und zwei Prozesse A und B das obige Programm ausführen, wie groß ist der gespeicherte Wert am Ende ?



Normalerweise ist (und das wird auch in der überwiegenden Anzahl der Fälle richtig sein) die Anwort 3. Aber man kann auch einen Fall konstruieren, wo diese Antwort nicht stimmt:

| Zeitpunkt | Programmschritt | Wert |
|---|---|---|
| 1 | Einlesen (A) | 1 |
| 2 | Inkrementieren (A) | 2 |
| 3 | Einlesen (B) | 1 |
| 4 | Inkrementieren (B) | 2 |
| 5 | Zurückschreiben (B) | 2 |
| 6 | Zurückschreiben (A) | 2 |

Das gleiche Problem kann bei Kernelcode auftreten, der von verschiedenen Prozessen „gleichzeitig“ verwendet wird. Dann muss ein „kritische Abschnitt“ gesichert werden (in unserem Fall dürfen die 3 Schritte nicht „auseinander gerissen“ werden). Die Kenntnis des Unterbrechungsmodells (siehe weiter untern) ist daher für den Kernelcode-Entwickler unabdingbar, um zu entscheiden, ob und wie ein kritischer Abschnitt geschützt werden muss.

## Speicherverwaltung und Virtuelle Adressierung:

Der virtuelle Speicher ist heute für das Multitasking die gängige Form der Speicherverwaltung. Hinter diesem Konzept stehen folgende Gedanken:

(1) Ein Prozess sollte auch dann ablaufen können, wenn er nur teilweise im Hauptspeicher ist. Wichtig ist hierbei, dass die Teile des Prozesses (Daten und Code) im physikalischen Speicher sind, die gerade benötigt werden.

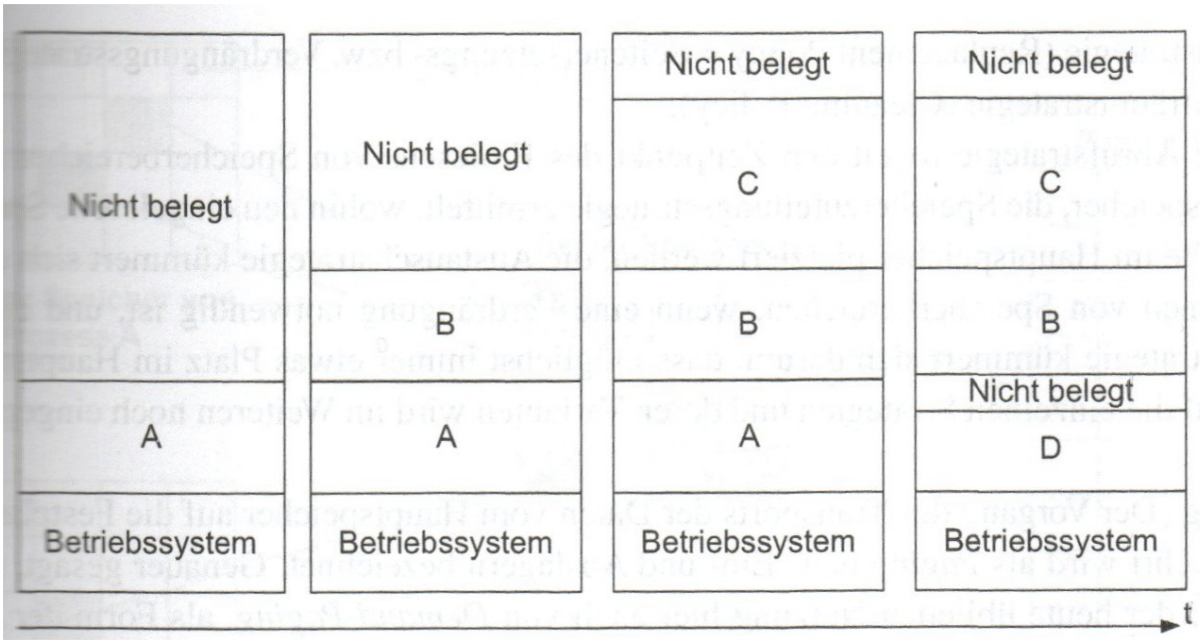
(2) Der Speicherbedarf eines Programms sollte größer als der physikalisch vorhandene Hauptspeicher sein können.

(3) Ein Programmierer sollte am besten nur einen kontinuierlichen (linearen) Speicherbereich, beginnend bei Adresse 0 sehen und sich nicht um die Zerstückelung (Fragmentierung) des Hauptspeichers auf mehrere Benutzer kümmern müssen.

Das Betriebssystem hält die gerade benutzten Speicherbereiche der Prozesse im Hauptspeicher und den Rest auf einem Sekundärspeicher (einer Festplatte). Man kann davon ausgehen, dass auch große Programme nicht den gesamten Speicher auf einmal benötigen, sondern sich im Code- und Datenbereich allermeist „lokal“ aufhalten (Lokalität des Programms und der Daten).

Nur in wenigen Fällen müssen Programmteile oder Daten vom Betriebssystem nachgeladen werden, eventuell wieder andere Bereiche von anderen Prozessen entfernt werden (**Swapping**). Swapping ist also die Auslagerung eines realen RAM-Speicherbereichs in einen anderen, sekundären Speicher.

Beim Wiedereinlagern kann es sein, dass ein anderer Adreßraum zur Verfügung steh: Das Betriebssystem muss daher eine Tabelle kennen, in der die zu verschiebenden Adressen des Programms festgehalten sind (Relocation-Table). Eine typische Situation für 4 Prozesse A - D zeigt das folgenden Bild:

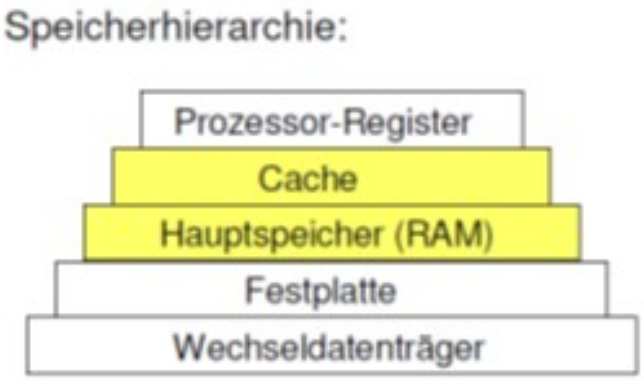



Der Begriff **Paging** taucht dagegen im Zusammenhang mit der **virtuellen Adressierung** auf und bezeichnet im engeren Sinn lediglich den Vorgang der Abbildung der virtuellen Adressen auf die physischen. Im weiteren (fehlerhaften) Sinne wird er synonym für Swapping benutzt.

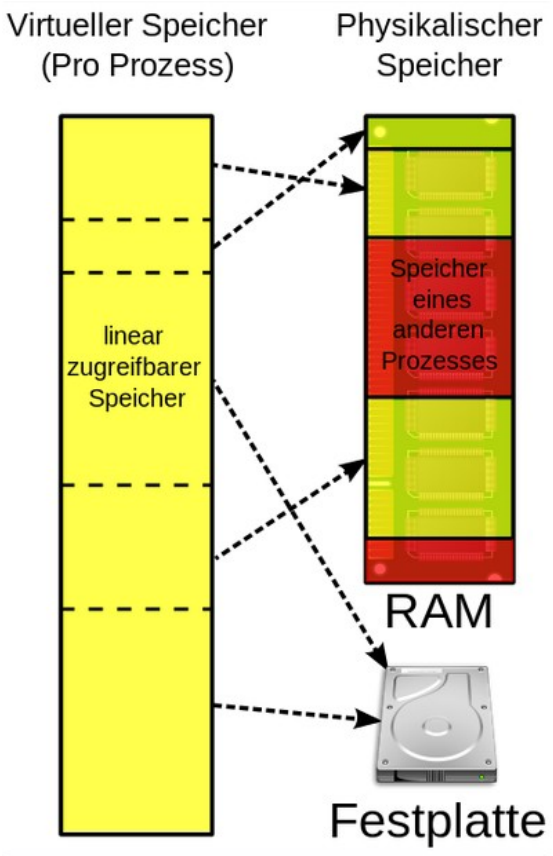
Die virtuelle Speicherverwaltung ist eine spezielle Speicher-verwaltung in einem Computer. Der virtuelle Speicher bezeichnet den vom tatsächlich vorhandenen Arbeitsspeicher unabhängigen Adressraum, der einem Prozess vom Betriebssystem zur Verfügung gestellt wird.

Das Konzept der virtuellen Speicherverwaltung entstand aus der Trennung in primäre Speichermedien, auf die der Prozessor direkt Zugriff hatte (meist Magnetkernspeicher), und sekundäre externe Speichermedien (meist Magnettrommelspeicher) in den Computern der 1950er Jahre und dem Wunsch, diese automatisch auf gleicher Ebene zu verwalten.

Das Konzept des virtuellen Speichers wurde 1956 vom Physiker Fritz-Rudolf Güntsch an der Technischen Universität Berlin entwickelt. Das folgende Bild zeigt eine typische Speicherhierarchie:



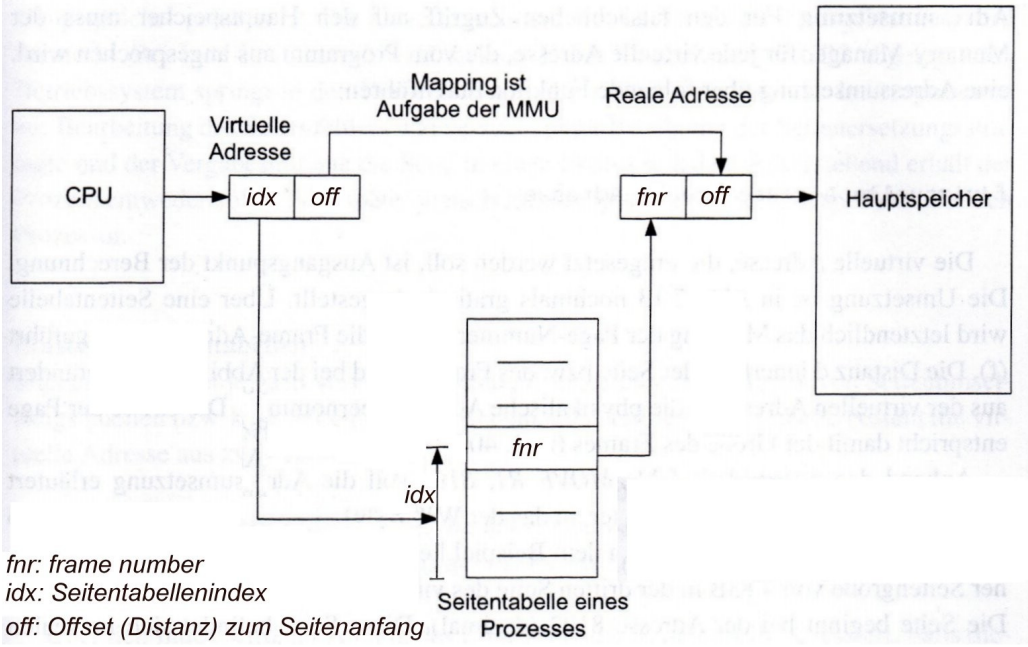

Das Konzept der virtuellen Adressierung, das in den 1950er Jahren entstand, wird heute von den meisten Prozessorarchitekturen unterstützt und in nahezu allen modernen Betriebssystemen verwendet. Für den Programmierer sichtbar sind die virtuellen Adressen, die dann in die physikalischen (oder physischen) Adressen übersetzt werden. Auch parallel laufenden Prozessen kann nun ein unzerstückelter (unfragmentierter) Adressraum (der meist mit der virtuellen Adresse 0 beginnt), zugewiesen werden:



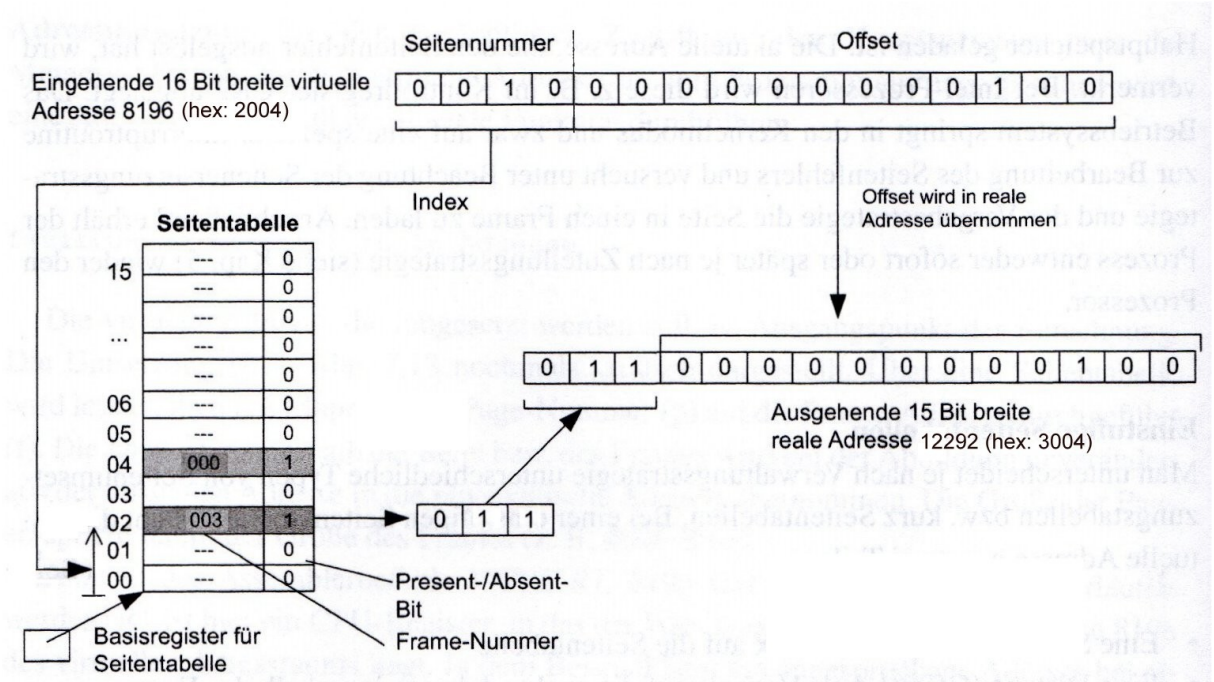

Bei dieser Zuordung werden die Speicherbereiche des virtuellen und des realen Adressraums in Blöcke aufgeteilt. Die Blöcke des virtuellen Adressraums werden als Seiten (Pages) und die des realen

Adressraums als Seitenrahmen (Page Frames, Frames, auch: Kacheln, oder kurz Rahmen genannt) bezeichnet. Eine ausgewogene Größe der Seiten bzw. Seitenrahmen ist wichtig, da bei zu kleinen Seitenrahmen mehr Ein-/Ausgabeoperationen erforderlich sind und bei zu großen Seiten Speicherplatz verschwendet wird. In vielen 32-Bit-Prozessoren wurde ein Größe von 4 kByte gewählt, aber auch andere Größen (1, 8, 16, 64 kByte und sogar 2, 4 und 16 Mbyte) sind vorzufinden.

Das Prinzip der Adressübersetzung zeigt das folgende Bild:




Ein konkretes Beispiel ist der folgenden Darstellung zu entnehmen:



Wichtige Begriffe:

**Virtuelle Adresse:** Diese ist sichtbar für das Programm / den Programmierer.

**Physikalische oder Physische Adresse**: diese wird an den Systembus angelegt und an den Speicher (und gegebenenfalls die IO-Einheiten) übermittelt.

Durch die virtuelle Adressierung werden auch die wichtigen **Speicherschutzmechanismen** realisiert; die Seitentabelle enthält auch Information darüber, ob ein Zugriff überhaupt gestattet wird. Falls nicht, erfolgt natürlich keine Übersetzung, sonder die MMU löst einen Interrupt aus. Man unterscheidert beim Speicherschutz:

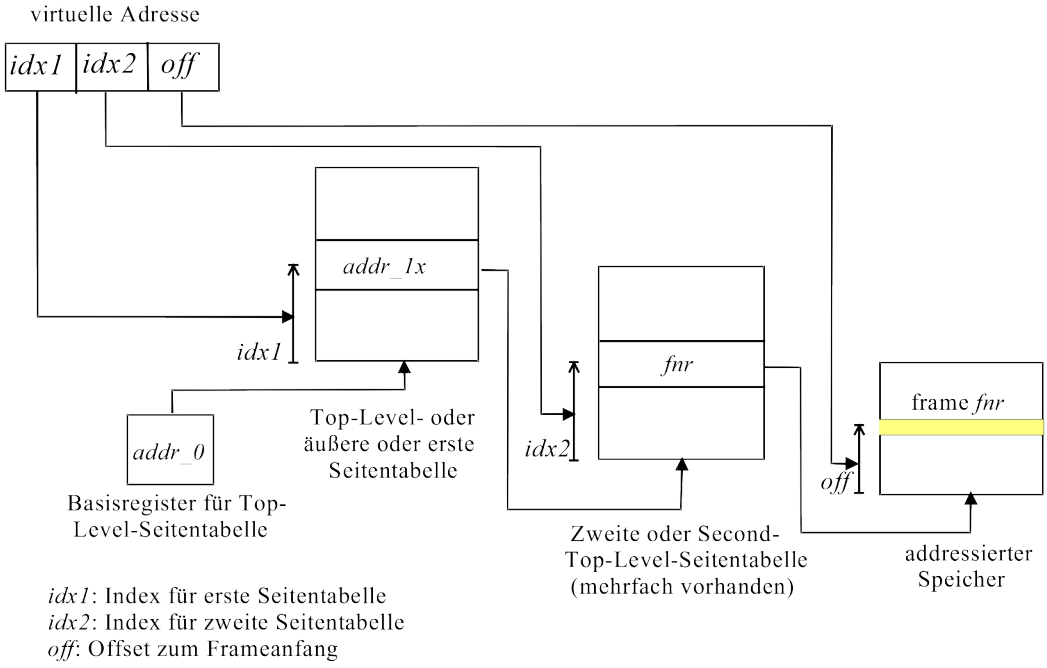
(1) Horizontale Trennung: Programme können nicht auf Speicher anderer Programme zugreifen.

(2) Vertikale Trennung: Das Funktionieren des Betriebssysiem darf nicht durch fehlerhafte Anwenderprogramme gefährdet werden.

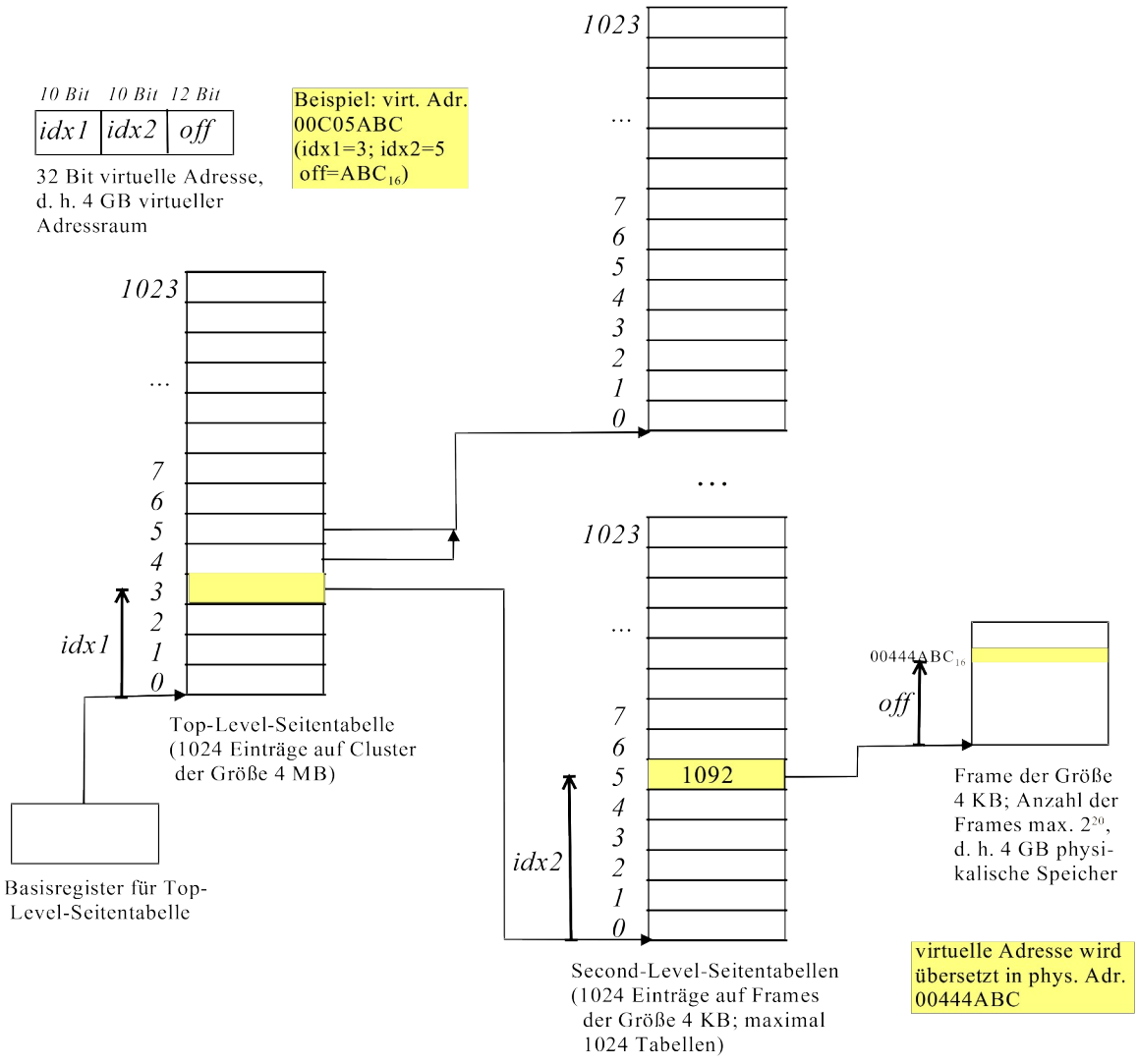
Ein Interrupt wird auch dann ausgelöst, wenn die aktuell benötigte Seite gerade nicht im Hauptspeicher vorhanden ist. Dann wird der sogenannte „Page-Fault-Handler“ des Betriebssystems aktiviert.

Für große Adressräume sind auch große Seitentabellen zu verwalten; z. Bsp. ergeben sich bei einer 32-Bit-Adresse und einer Seitengröße von 4 kByte sind immerhin schon eine Million ($2^{20}$) Einträge in der Seitentabelle, und dies für jeden einzelnen Prozess, der gerade im System ist. Man würde durch die Verwaltung großer Seitentabellen viel Speicher verschenken.

Das Prinzip der zwei-stufigen Adressverwaltung, das auch auf mehr Stufen angewandt werden kann, zeigt die nächste Abbildung:




# Ein konkretes Beispiel zeigt das folgende Bild:



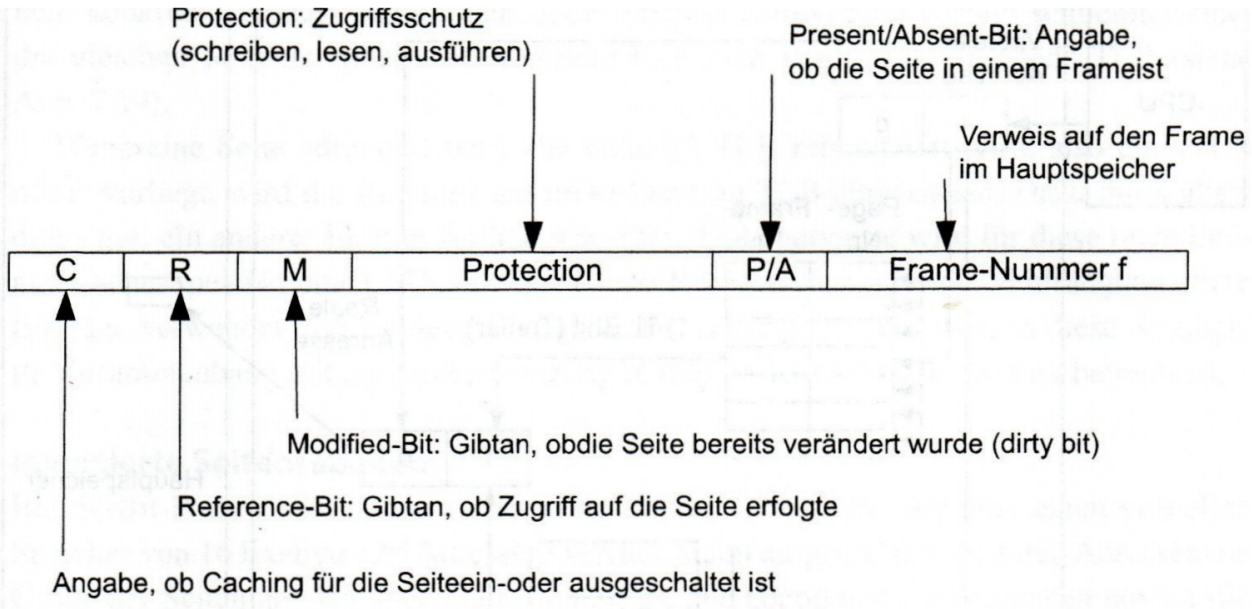
Hier wird die virtuelle Adresse aufgeteilt in die Indices *idx1* (10 Bit), *idx2* (10 Bit) und den Offsett-Anteil *off* (12 Bit, d. h. Seitengröße ist 4 Kbyte). Die äußere Seitentabelle benötigt dann nur noch $2^{10}$ Einträge, die Anzahl (und Größe) der inneren Seitentabellen ist abhängig von dem tatsächlich benötigten Adressraum des Prozesses.

Linux verwaltet seit der Kernel-Version 2.6.11 standardmäßig eine vierstufige Scitentabelle, bis zu dieser Version wurde eine dreistufige Seitentabelle verwendet. Dabei ist ein Mapping der in Hardware implementierten Mechanismen auf die Betriebssystemmechanismen erforderlich.

Zu der Verwaltung der Seitentabellen gehören natürlich auch die Mechanismen, um eine nicht im Hauptspeicher eingelagerte Seite bei einem Seitenfehler zu laden und dafür den Hauptspeicher möglichst immer von unnötigen Ballastseiten zu befreien. Trotz des immensen Aufwands ist die virtuelle Speieherteehnik mit „Demand-Paging“ heute das am meisten verwendete Verfahren zur Speicherverwaltung.

Es sei hier erwähnt, dass die Speicherbereiche für die Seitentabellen je nach Betriebssystem auch dem Paging unterliegen können und daher bei Bedarf ausgelagen werden. Ein Prozess kann allerdings nur aktiv werden, wenn er Zugriff auf die Seitentabelle im Hauptspeicher hat.

Zum Abschluss ist ein typischer Seitentabellen-Eintrag dargestellt; man erkennt, dass nicht nur die Frame-Nr hinterlegt ist, sondern noch weitere Informationen, auf die wir hier nicht näher eingehen:



## Das System-Call-Interface:

Das Betriebssystem ist von den Anwendungsprozessen abgeschottet (Konzept der virtuellen Maschine). Anwendungsprogramme nutzen die Dienste des Betriebssystems über eine Zugangsschicht. Der Aufruf der Dienste erfolgt über sog. Systemcalls.

Ein Systemecall ist ein Dienstaufruf an das Betriebssystem, bei dessen Ausführung in den Kernelmodus gewechselt wird. Der Kontrollfluss wird von einem Anwendungsprogramm an den Kernel übergeben.

In jedem Mehrzweckbetriebssystem wird üblicherweise eine Bibliothek mit Funktionen bereitgestellt, die höherwertige Dienste anbietet, um den Übergang vom Benutzermodus in den Kernelmodus möglichst komfortabel zu gestalten. Unix und Linux stellen hierfür eine Bibliothek, die meist als C-Library oder `clib` bezeichnet wird, bereit. Beispiele für typische Systemaufrufe unter Linux sind:

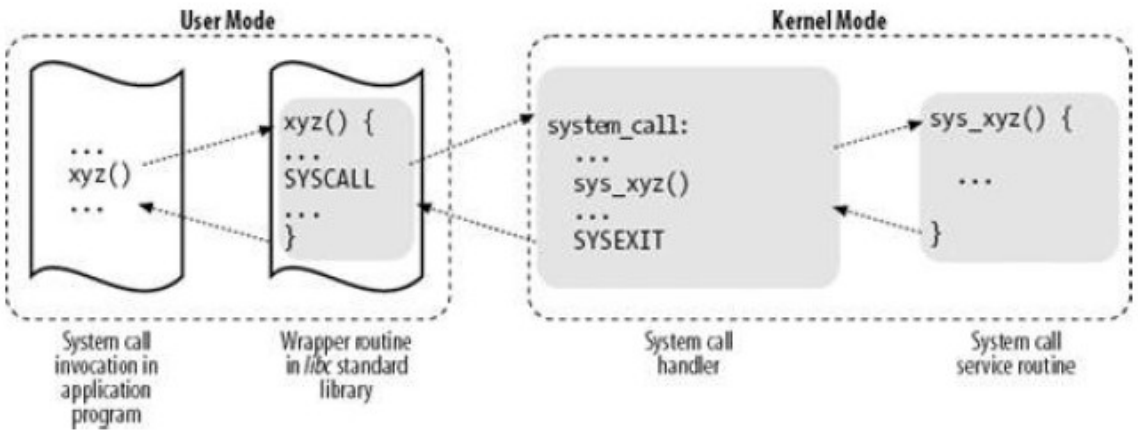
- `fork` zur Prozesserzeugung
- `exit` zum Beenden eines Prozesses
- `open` zum Öffnen einer Datei
- `close` zum Schliessen einer Datei
- `read` zum Lesen einer Datei
- `write` zum Schreiben einer Datei

Damit die Anwendungsprogramme die Adressen von Systemroutinen nicht kennen müssen, wird ein spezieller Aufrufmechanismus verwendet, der auch als Trap bezeichnet wird. Ein Trap ist ein **Software-Interrupt**, der für einen Übergang vom Benutzermodus in den Kernelmodus sorgt, um den aufgerufenen Systemdienst auszuführen. Software-Interrupts werden üblicherweise durch einen speziellen Masehinenbefehl des Prozessors oder durch den Interruptcontroller unterstützt. Bei Ausführung des Systemcalls über einen Software-Interrupt wird wie folgt verfahren:

- Der aktuelle Kontext des laufenden Programms, also die Information, welehe den aktuellen Status eines Prozesses beschreibt, wird gesichert.
- Der Program Counter wird mit der Adresse der passenden Systemroutine belegt.

- Vom Benutzermodus wird in den Kernelmodus geschaltet.
- Die adressierte Systemroutine wird durchlaufen.
- Anschließend wird wieder der alte Kontext des Prozesses hergestellt und der Programcounter mit der Adresse des Befehls nach dem Systemcall belegt.

Das folgenden Bild veranschaulicht diesen Mechanismus:




**Aus: Understanding the Linux Lernel**

Der Trap-Mechanismus hat den Vorteil, dass dcr Aufruf eines Systemcalls von einem Anwendungsprogramm aus ermöglicht wird, ohne dass die tatsächliche Adresse der Sysstemroutine bekannt sein muss (Information Hiding). Es wird nur eine Nummer der gewünschten Funktion als Parameter übergeben. Alle Systemcalls zusammen bilden die Schnittstelle der Anwendungsprogramme zum Betriebssystemkern (Kernel).

Bei Linux hängen die Systemcalls von der Architektur ab und finden sich z. Bsp. in der Datei
`</arch/arm/include/generated/uapi/asm/unistd-common.h>`

In der Architektur ARM ist in der aktuelle Linuxversion Platz für 400 Systemcalls: `<arch/arm/include/generated/asm/unistd-nr.h>`

Zusammenfassung:

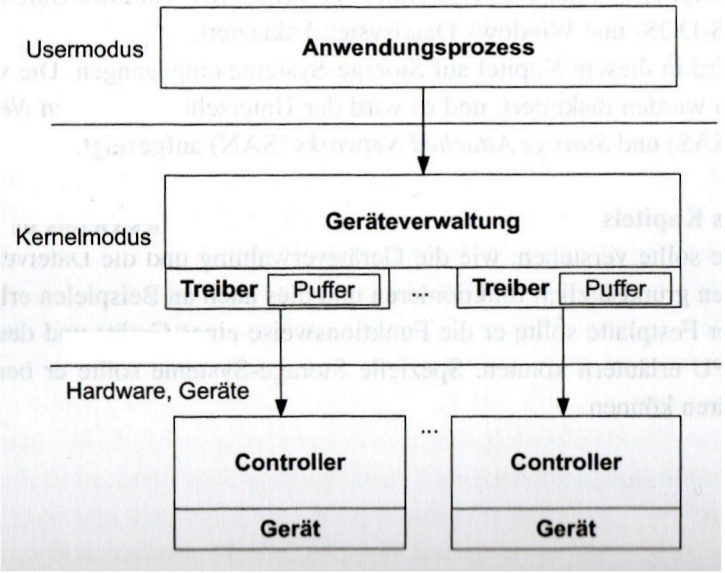
- Der Aufruf einer Betriebssystem-Funktion durch eine Anwendung (z.B. File-I/O, Hauptspeicher-Anforderung, Starten eines anderen Programms, ...) erfolgt über das Systemcall-Interface.

- Wie bei einem Funktionsaufruf werden bei einem Systemcall Parameter-Werte übergeben, und der Aufrufer bekommt einen Rückgabewert. Danach arbeitet das Programm normal weiter.
- **Technisch ist es aber kein CALL-Maschinenbefehl**, sondern eigener Befehl “SYSCALL” oder “SWI” (Software-Interrupt) (mit entsprechendem SYSRET- oder IRET-Befehl, der alles wieder zurückschaltet und in die Anwendung zurückkehrt)
- Es passiert viel mehr als bei einem normalen Funktionsaufruf: Der Ausführungs-Modus wird auf Kernelmodus umgeschaltet (Kontextwechsel mit Umprogrammierung der MMU, Kernel-RAM ist sichtbar). Der komplette Register-Zustand der Anwendung wird gesichert. Eine weitere Unterbrechung vor Beendigung des zum System Call gehörenden Kernel-Codes ist möglich.
- Ein Systemcall ist viel langsamer als ein Funktionsaufruf: Einige 100 bis 1000 Takte statt weniger als 10 Takte!

## I/O-Management:

Die Geräteverwaltung, auch Ein- und Ausgabeverwaltung oder I/O-Mananagement genannt, dient als Teil des Betriebssystems der optimierten Verwaltung von externen Geräten zur Ein- und Ausgabe. Zu den Geräten gehören u. a. Festplatten, USB-Controller und USB-Geräte, CD-ROM- und DVD-Laufwerke, die Tastatur, der Bildschirm und die Netzwerkanbindungen.

Die Leistungsfähigkeit eines Computersystems hängt nicht nur vom Prozessor, sondcrn auch von der Geschwindigkeit der Datenübertragung zwischen dem Prozessor, dem Hauptspeicher und den Ein- und Ausgabegeräten ab. Das nächste Bild zeigt die Schichtung der Ein/Ausgabeverwaltung:




Die Geräteverwaltung stellt die Schnittstelle zwischen den physikalischen Geräten und dem Rest des Betriebssystems dar und hat die Aufgabe, die Geräte anzusteuern. Hierzu gehört die Ausgabe von Kommandos an externe Geräte, die Interruptbearbeitung und eine entsprechende Fehlerbehandlung.

Die einzelnen Geräte sind von ihrer Funktionalität und ihrem Aufbau her sehr unterschiedlich. Daher ist oft die hardwarenahe Funktionalität in einer eigenen Schicht gekapselt.

Gerätespezifische Programmteile werden werden in Softwarekomponenten realisiert, die auch als Gerätetreiber bezeichnet werden. Treiber laufen typischerweise im Kernelmodus ab.

Es gibt im Wesentlichen drei Aufgaben für das I/O-Management:

• Ein Interface zur systemkonformen Integration von Hardware zur Verfügung stellen

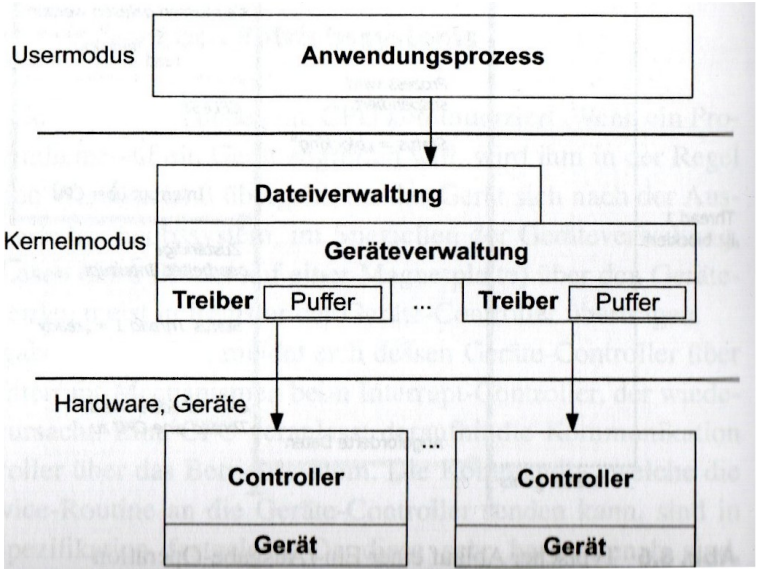
• Eine einheitliche Programmierschnittstelle für den Zugriff auf die Peripherie zur Verfügung stellen

• Ordnungsstrukturen für Daten in Form von Verzeichnissen und Dateien über so genannte Dateisystem (file system) zu realisieren.

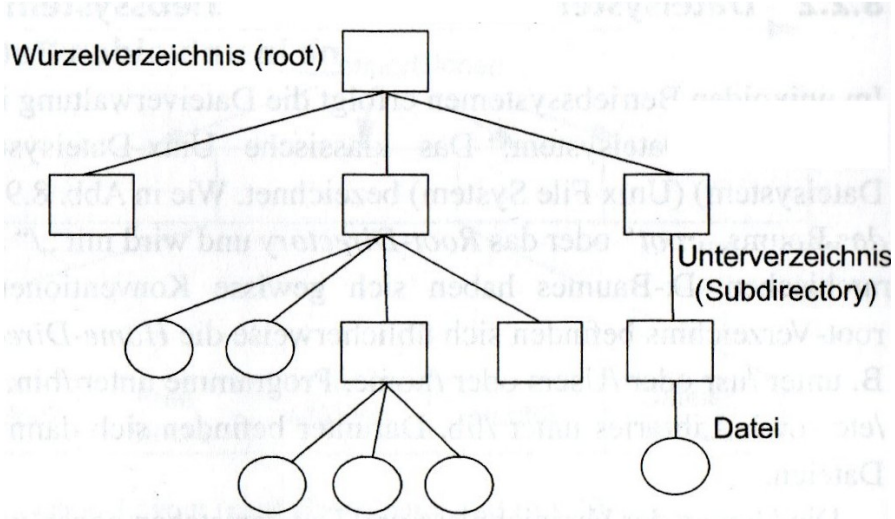
Dateisysteme:

Persistent, also dauerhaft zu verwaltende Daten werden üblicherweise in Dateien (Files) gespeichert, die auf externen Medien wie HDDs oider SSDs abgelegt werden. Eine Datei ist ein abstrakter Mechanismus zur Speicherung und zum Finden von Informationen.

Der Teil des Betriebssystems, der sich mit Dateien und deren Organisation befasst, wird als Dateiverwaltung bezeichnet. Die Dateiverwaltung liegt von der Schichtung her über der Geräteverwaltung und nutzt die Dienste der Geräteverwaltung zum Zugriff auf die Hardware.



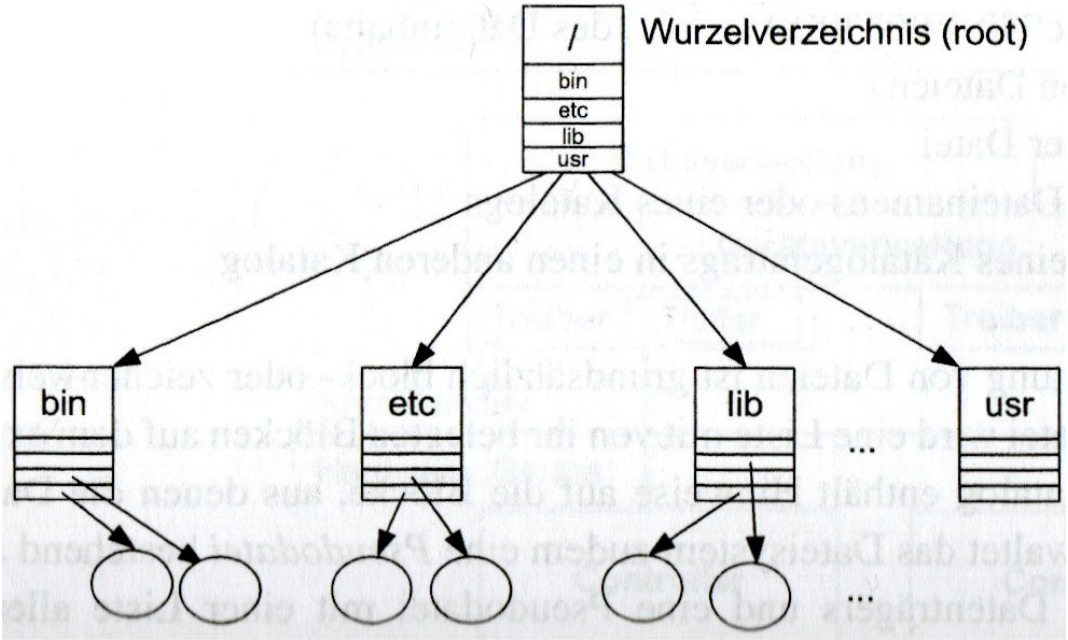

Dateien werden in sog. Dateisystemen (file system) verwaltet. Dateisysteme werden meist hierarchisch organisiert, wobei Gruppen von Dateien in Dateiverzeichnissen (directory) zusammengefasst werden können. Jedes Verzeichnis kann wieder Unterverzeichnisse (subdirectories) oder die Dateien selbst beinhalten. Durch die hierarchische Anordnung entsteht ein (ungerichteter) Verzeichnisbaum:



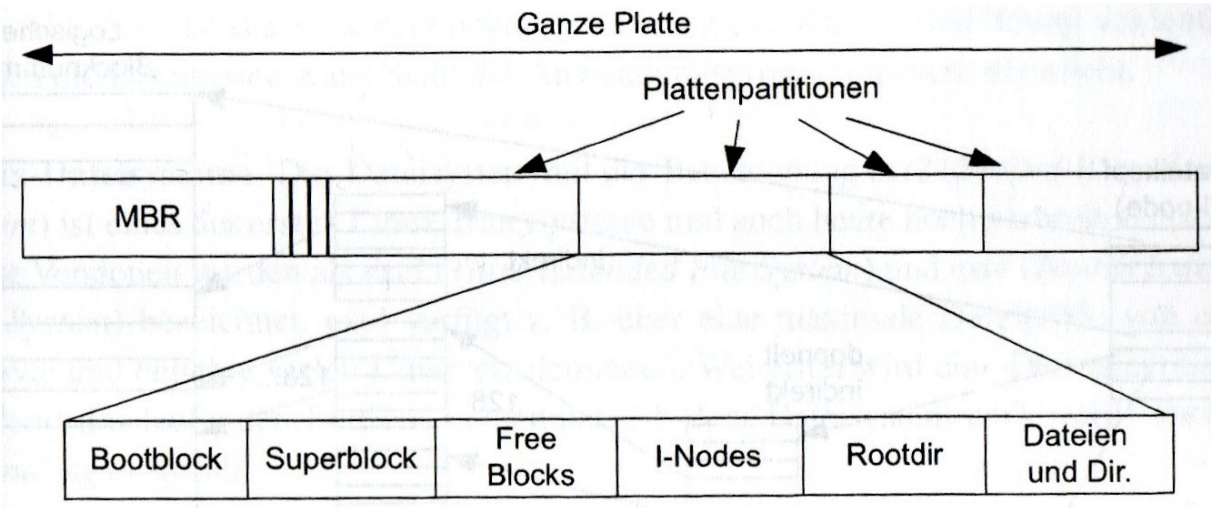
Für die Bearbeitung von Dateien und Verzeichnissen stellen Betriebssysteme bestimmte Systemdienste über Systemcalls zur Verfügung. Hierzu gehören:

- Erzeugen eines Katalogeintrags (directory)
- Löschen eines Katalogeintrags (inkl. des Dateiinhalts)
- Kopieren von Dateien
- Löschen einer Datei
- Ändern des Dateinamens oder eines Katalogs
- Übertragen eines Katalogeintrags in einen anderen Katalog

Unix / Linux verfügt über ein systemweites Dateisystem. Die Wurzel des Baumes heisst root oder root-Directory und wird mit / abgekürzt:




Ein Dateisystem hat unter unixoiden Betriebssystemen ein vorgegebenes Layout, das die folgenden Bestandteile enthält:

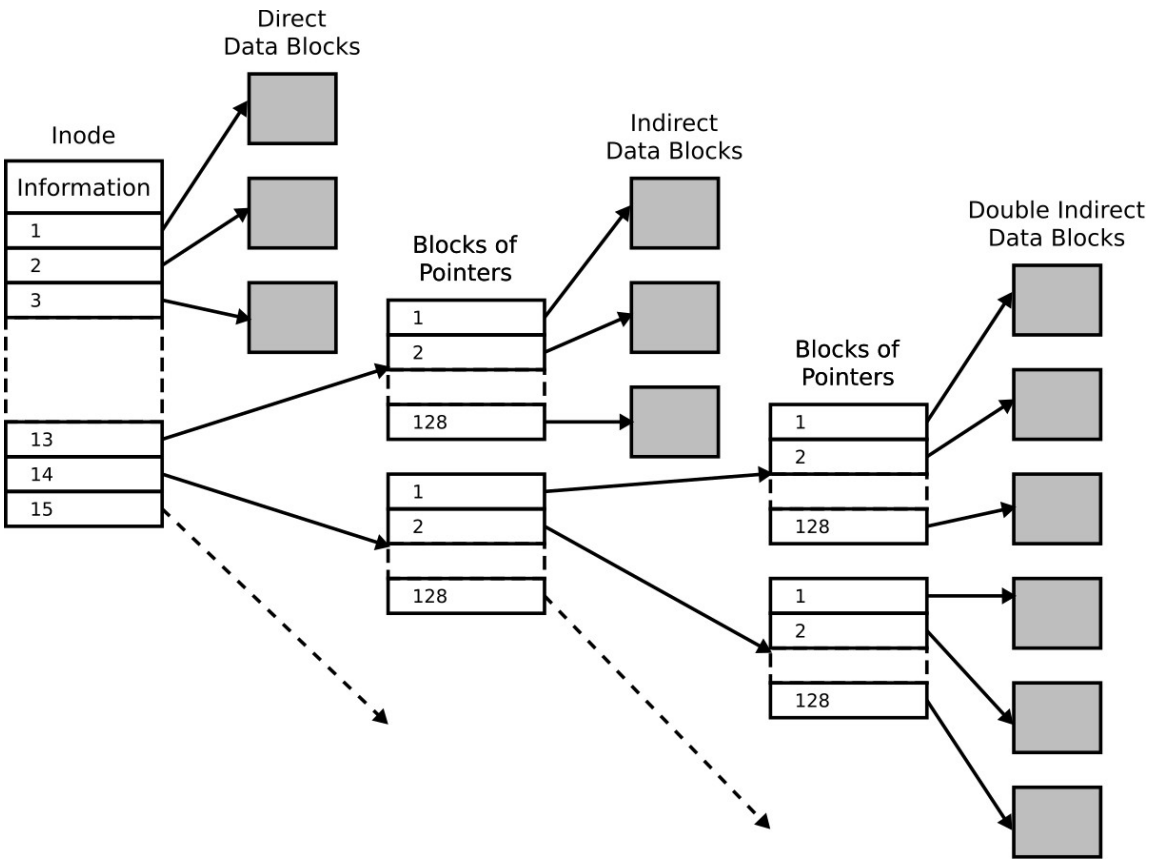



Der Master Boot Record (MBR) auf Sektor 0 zum Booten des Rechnersystems.

Am Ende des MBR liegt die Partition Table: Eine Platte kann i. d. R. in mehrere Parttionen eingeteilt werden.

- Der Bootblock wird beim Hochfahren gelesen und ausgeführt.
- Der Superblock enthält Verwaltungsinformationen zum Dateisystem wie etwa die Anzahl der Blöcke.
- In den sog. Free Blocks werden die freien Blöcke des Dateisystems verwaltet.
- Rootdir enthält den Inhalt des Dateisystems (nach der Wurzel).
- i-Nodes

Ein Inode (englisch index node, gesprochen „eye-node“) ist die grundlegende Datenstruktur zur Verwaltung von Dateisystemen mit unixartigen Betriebssystemen. Jeder Inode wird innerhalb eines Dateisystems eindeutig durch seine Inode-Nummer identifiziert. Jeder Namenseintrag in einem Verzeichnis verweist auf genau einen Inode. Dieser enthält die Metadaten der Datei und verweist auf die Daten der Datei beziehungsweise die Dateiliste des Verzeichnisses. Diese Verweise können auch mehrstufig sein, so dass erst die Einträge auf der letzten Ebene auf die Dateiblöcke verweisen:




Das Dateisystem mit der Bezeichnung *ext2* (Secondary Extended File System) ist eines der ersten Linux-Dateisysteme und auch heute noch verbreitet. Die nachfolgenden Versionen sind *ext3* und *ext4*. Letzteres verfügt über eine maximale Dateigröße von einem Exabyte (Milliarde Gigabytes) und beliebig vielen Unterverzeichnissen. Weiterhin wird eine Defragmentierung im laufenden Betrieb unterstützt. Ab dem Dateisystem *ext3* wurde auch ein Journaling eingeführt. Dabei werden alle Änderungen auch in Metadaten mitgeschrieben mit dem Zweck, das Dateisystem und einzelne Dateien bei einem Systemausfall wieder herstellen zu können.

Die Abstraktionschicht oberhalb der eigentlichen Dateisysteme wird als Virtuelles Dateisystem (VFS) bezeichnet. Es bietet eine einheitliche API für unterschiedliche Dateisysteme. Es können hier aber auch Dateiverzeichisse eingebunden werden, die keine eigentlichen Dateien auf einem Datenträger repräsentieren wie virtuelle Dateisysteme oder Gerätedateien.

## Virtuelle Dateisysteme

Verzeichnisse, die keine eigentlichen Dateien auf einem Datenträger verwalten, können ebenfalls in den Dateibaum eingebunden sein. Diese Verzeichnisse gehören gehören zum so genannten virtuellen Dateisystem. Sie dienen als Schnittstelle zum Kernel und können für den Datenaustausch mit diesem verwendet werden. In Linux gibt es zwei wichtige virtuelle Dateisysteme:

### procfs

Das `/proc` Filesystem ermöglicht es, Informationen über Kernel-subsysteme, wie zum Beispiel Speichernutzung, angeschlossene Peripherie usw. zu erhalten. Ebenfalls kann das Verhalten des Kernels teilweise gesteuert werden, ohne dass die Quellen neu kompiliert werden müssen. Weiter ist es darüber möglich, Kernelmodule zu laden oder einen Neustart des Systems auszulösen.

### sysfs

Das `/sys` Filesystem ist ein virtuelles Filesystem, mit dem Informationen über Kernelobjekte in den Userspace übergeben werden können. Dabei können nicht nur Informationen über Geräte und Treiber abgerufen werden, sie können darüber auch aus dem Userspace konfiguriert werden. *sysfs* ist im Gegensatz zu *procfs* stark hierarchisch geschachtelt und nicht dazu ausgelegt von Personen direkt gelesen zu werden. Im *sysfs* gibt es auch rein binäre Schnittstellen, das heisst, sie liegen nicht mehr in ASCII Textform vor.

Wird ein Geträtetreiber hinzugefügt, kann er einen entsprechenden Knoten im *sysfs* anlegen. Diese Aktion macht das Gerät im Userspace sichtbar und löst ein Kernelevent aus, das einen im Userspace laufenden Dämon (*udev*) darüber informiert. Der *udev*-Dämon kann dann so konfiguriert werden, dass die passende Gerätedatei im */dev*-Verzeichnis automatisch erzeugt wird.

Beim Entfernen eines Gerätes verschwindet der Device-Node im *sysfs*, und *udev* sorgt dafür, dass die Gerätedatei wieder entfernt wird.

Gerätedateien:

Idee bei Unix/Linux des einheitlichen Zugriffs auf die Peripherie ist die, dass auch Geräte durch Dateien im Dateisystem repräsentiert werden („Everything is a file“).

Diese Gerätedateien sehen wie herkömmliche sonstige Dateien aus, enthalten aber keine Vielzahl von Daten, sondern letztlich nur eine Kennung, welcher Gerätetreiber bei einem Zugriff auf diese Datei angesprochen werden soll. Gerätedateien sind durch ein Attribut gekennzeichnet und so von regulären Dateien unterscheidbar:

```
-rw-r----- 1 root adm  2466   Mar  9 11:00 /var/log/syslog
crwxrwxrwx 1 root root 242, 0 Mar  6 21:17 /dev/led3
```

Bei Linux wird zwischen drei Geräteklassen unterschieden. Es gibt Zeichengeräte (**char devices**), wie zum Beispiel eine serielle Schnittstelle, Blockgeräte (**block devices**), die über ein Dateisystem verfügen und Netzwerk Schnittstellen (**networking devices**) für den Austausch von Daten zwischen Systemen. Der Unterschied zwischen Zeichen- und Blockgeräten liegt in der Verwaltung der Daten im Kernel.

Netzwerk Schnittstellen repräsentieren normalerweise an das System angeschlossene Hardware, können aber auch ein reines Software-Gerät sein. Netzwerkschnittstellen unterscheiden sich komplett zu Zeichen- und Blockgeräten und können nicht über eine Datei angesprochen werden.

## Gerätetreiber:

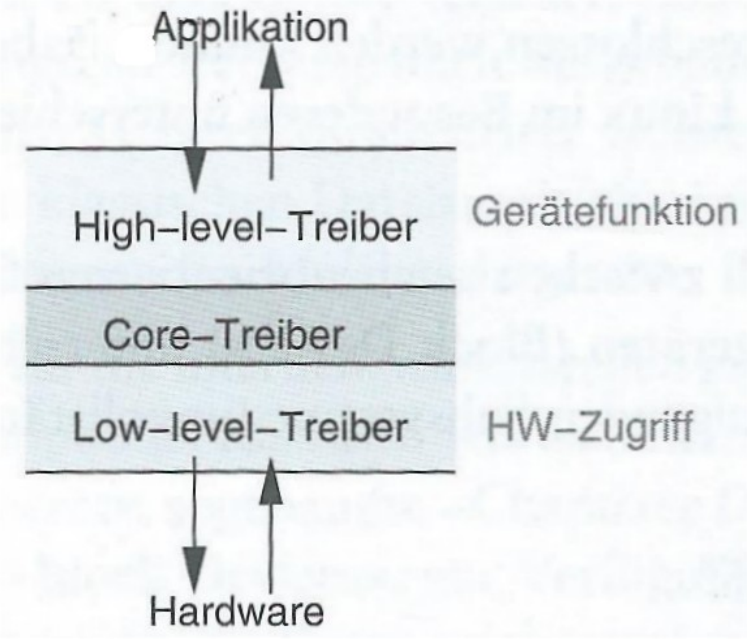
Kernel-Code der spezifisch für ein bestimmtes Gerät entwickelt wurde, heißt **Gerätetreiber**. Nicht dazu zählen Prozessor und Speicher, da spricht man von architekturspezifischem Code

Erst der Gerätetreiber macht es einer Applikation möglich, über ein festgelegtes Interface die Funktionalität eines Gerätes zu nutzen.

Da die verschiedensten Geräte über diverse Bussysteme (z. Bsp. PCI, SCSI, USB, I2C) angeschlossen werden können, haben Betriebssysteme im Allgemeinen und Linux im Speziellen unterschiedliche Treiber-Subsysteme, die die klassische „Liste“ von zeichen- und blockorientierten Geräten erweitern:

- Character Devices
- Block Devices
- Netzwerk
- USB (universal Serial Bus)
- Bluetooth
- FireWire (IEEE1394)
- SCSI (Small Computer System Interface)
- GPIO
- IrDA (Infrared Data Association)
- Cardbus und PCMCIA
- I2C (serielles Kommunikationsprotokoll)
- SPI (serielles Kommunikationsprotokoll)

In Linux passt für viele Treiber passt ein Schichtenmodell, das zwischen Low-Level- und High-Level-Driver unterscheidet. Dazwischen liegt oft noch eine Schicht mit dem so genannten Core-Treiber. Man nennt diese Treiber deshalb auch geschichtete Treiber (stacked driver).




Der Low-level-Treiber greift direkt auf die Register der Hardware zu. Der High-Level-Treiber ist dagegen für einen Gerätetyp (z. Bsp. eine

Webcam) zuständig. Bei USB werden zwischen Geräte und Treiber Kommandopakete verschickt. Im Fall von USB stellt der Core-Treiber Funktionen zum Geräter/Treiber-Management zur Verfügung.

## 1.3. Modulkonzept bei Linux

Linux basiert auf einem monolithischen Kernel; das bedeutet, dass alle Aufgaben des Betriebssystems wie Speicher- und Prozessverwaltung im priviligierten Kernel-Modus ablaufen.

Für die Entwicklung von Treibern hat ein monolithischer Kernel zwei Nachteile:

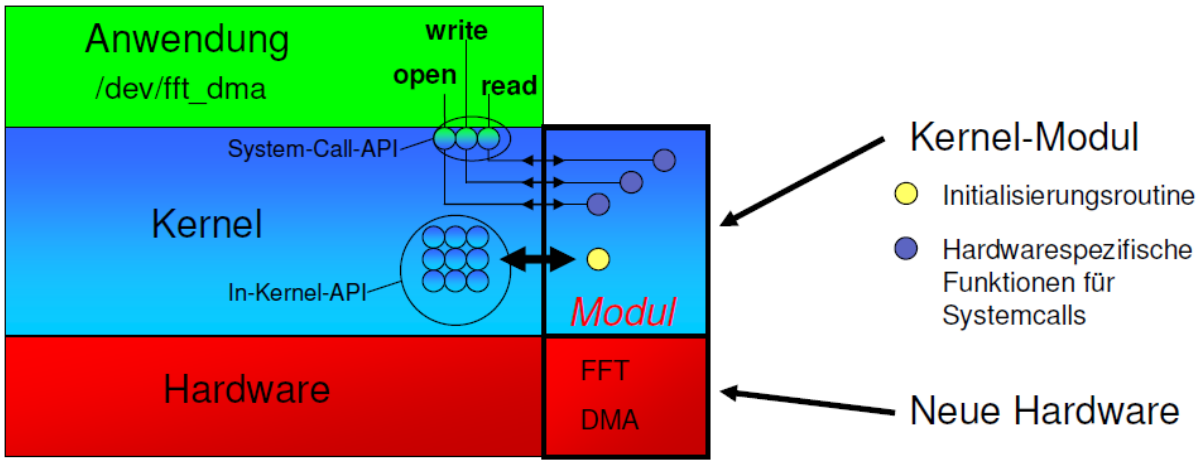
- Der erstellte Treibercode hat alle Zugriffsrechte des Kernels. Fehler können daher zum Absturz des Gesamtsystems führen.
- Bei jeder neuen Treiberversion muss der Kernel neu gelinkt und neu gestartet werden.

Den zweiten Nachteil vermeidet Linux durch ein Modulkonzept: Neu erstellter Kernelcode kann zur Laufzeit des Kernels zum Kernelcode dazugefügt (und auch wieder entfernt) werden.

Sind Gerätetreiber integraler Bestandteil des Betriebssystemkerns, so heißen sie **Built-in-Treiber** oder auch **Kerneltreiber**. Sie werden zusammen mit dem sonstigen Kernelcode compiliert und stehen nach Start des Kernels zur Verfügung.

Dagegen können die **Modul-Treiber** zur Laufzeit des Kernels geladen und auch wieder entfernt (entladen) werden.

Bei der Entwicklung eines Kernel-Modul muss auf die **In-Kernel-API** zugegriffen werden: Diese Schnittstellen sind nicht an das System-Call-Interface angebunden und daher vom Userspace aus nicht erreichbar.

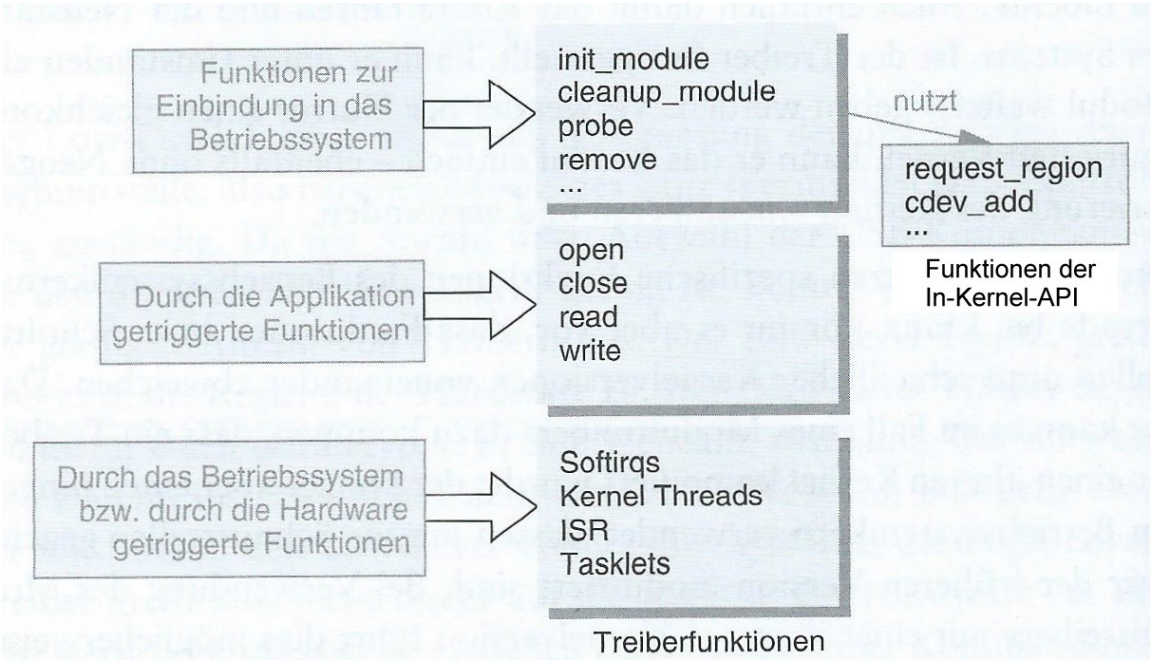



Daher muss die Kernel-Symbol-Tabelle (Einsprungadressen der Kernelfunktionen) beim Build-Prozess bekannt sein: Die Kernelquellen (insbesondere die Header-Dateien) müssen bei der Erstellung eines Moduls vorhanden sein. Für die Kompilierung des Kernelmoduls wird das Kernel-Build-System verwendet. Daher passt das Modul auch genau zu dieser Kernelversion. Kompatibilität zu neueren Kernelversionen ist nicht sichergestellt. Die In-Kernel-API könnte sich ändern. Grundsätzlich gilt: der Anwender benötigt einen Modultreiber, der genau zu seiner Kernelversion passt.

Funktionen, die ein Modul-Entwickler zu kodieren hat, sind:

1. Funktionen, die zur Einbindung des Moduls in den Kernel notwendig sind
   (oben gelb dargestellt)
2. Funktionen, die durch die Applikation angestoßen (getriggert) werden.
   (oben blau dargestellt)
3. Funktionen, die als Folge von Hardware-Interrupts angestoßen werden
   (oben nicht vorhanden)

Diese drei Arten von Funktionen sind nochmals in der nächsten Grafik dargestellt:




## 1.4. Abarbeitungskontext und Unterbrechungsmodell

Wir haben gesehen, dass bei einem Kontextwechsel die Register der CPU und MMU umgeladen werden und somit auch neue Seitentabellen für die Adreßumsetzung gültig werden

Man unterscheidet in Linux prinzipiell drei Abarbeitungskontexte:

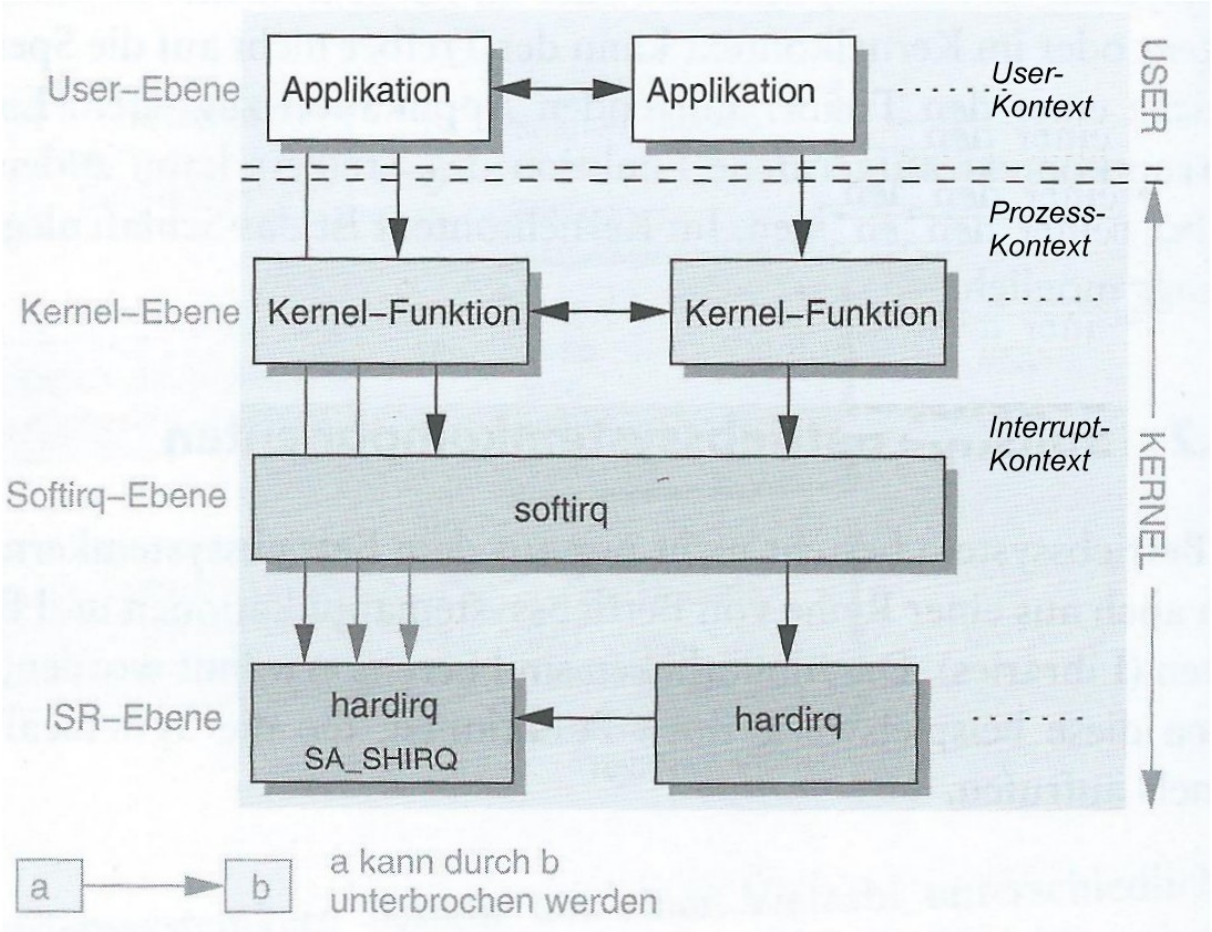
Eine Applikation läuft im **User-Kontext**. Die MMU ist so programmiert, dass die zugewiesenen Speicherseiten für die Applikation sichtbar sind.

Nach einem Systemcall wird der **Prozess-Kontext** erreicht. Die MMU ist umprogrammiert, so dass der Kernelspeicher sichtbar ist. Es ist noch die Information über den aufrufenden Prozess vorhanden (in Linux gibt es dazu die globale Variable `current`). Der Kernel hat noch Zugriff auf die nun inaktiven Seitentabellen des aufrufenden Prozesses und kann Daten zwischen dem Userspace und dem Kernelspace transferieren (siehe später Makro `put_user`, `get_user` etc.).

Nach jedem Hardwareinterrupt wird der **Interrupt-Kontext** erreicht. Der Zugriff auf Speicherbereiche im Userbereich ist hier nicht

möglich. Eventuell sind auch unmittelbar nach dem Wechsel in diesen Kontext nicht alle Speicherseiten des Kernels und damit auch nicht alle Funktionen verfügbar. Weiterhin dürfen in diesem Kontext keine Funktionen aufgerufen werden, die direkt oder indirekt blockieren können. Der Interrupt-Kontext soll stets so schnell wie möglich wieder verlassen werden.

Bezüglich der Unterbrechbarkeit gibt des bei Linux das folgenden **Schichtenmodell**, bei dem der im Interruptkontext laufende Code nochmals in die Soft-IRQ-Ebene und die ISR-Ebene („Hard-IRQ“) unterteilt ist. Code der Soft-IRQ-Ebene kann unmittelbar nach einem Sytem-Tick (Interrupt des Systemtimers) angestoßen werden, da er mit hoher Priorität (vor jedem Prozess und vor jedem Code im Prozesskontext!) abgearbeitet wird.




Das Schichtenmodell ist so aufgebaut, dass Code auf einer Schicht immer nur von Code einer darunter liegenden Schicht unterbrochen werden kann. Je tiefer die Schicht, desto höher die Priorität.

**User-Ebene:**
Code normaler Rechenprozesse (Tasks und Threads) läuft auf der User-Ebene. Er kann jederzeit unterbrochen werden oder selbst einen Interrupt auslösen, wenn z. Bsp. Dienste über das System-Call-Interface angefordert werden.

**Kernel-Ebene:**
Dem Code auf Kernelebene stehen sämtliche Funktionen des Betriebssystemkerns zur Verfügung, auch die Funktionalität, sich schlafen zu legen. Applikationsgetriggerte Funktionen werden in dieser Ebene abgearbeitet. Daten zwischen Applikation und Kernel (zwischen Userspace und Kernelspace) können übertragen werden.

**Soft-IRQ-Ebene:**
Codestücke der Soft-IRQ-Ebene sind allein durch Hardware-Interrupts unterbrechbar. Es dürfen keine Funktionen aufgerufen werden, die sich schlafen legen (also sich selbst unterbrechen). Der Zugriff auf Speicherbereiche einer Applikation sind nicht möglich. Routinen, die auf dieser Ebene ablaufen, sind Softirqs, Tasklets und Timer.

**ISR-Ebene:**
Auf dieser Ebene laufen die Interrupt-Service-Routinen. Dieser Code ist normalerweise nicht unterbrechbar. Auch hier sind wir im Interruptkontext (Schlafen verboten, kein Zugriff auf Speicherbereiche einer Applikation). Die ISR-Routine sollte so kurz wie möglich gehalten werden.

<u>Zusammenfassung:</u>

- Die gleiche Adresse (Zeiger) zeigt in unterschiedlichem Abarbeitungskontext auf verschiedene Speicherzellen im physikalischen Speicher ! (Oder ist in einem Kontext gültig, im anderen aber ungültig)

- Im Kernelkontext wird unterschieden zwischen dem Prozesskontext und dem Interrupt-Kontext. Der Prozesskontext wird geladen, wenn ein Prozess über einen System-Call einen Dienst des Betriebssystems anfordert. Der Kernelcode kennt die Seitentabellen des aufrufenden Userprogramms und kann einen

im Userkontext gültigen Zeiger auf einen Zeiger im Prozesskontext umrechnen, der auf die gleiche physikalische Zelle zeigt.

- Der Interruptkontext wird nach jedem Hardware-IRQ (IRQ = interrupt request) oder Software-IRQ (getriggert vom System-Timer) erreicht. Es stehen hier nicht alle Dienste des Betriebssystemkerns zur Verfügung. Es können keine Funktionen aufgerufen werden, die sich direkt oder indirekt schlafen legen. Der Zugriff auf Speicherbereiche im Userbereich ist ebenfalls nicht möglich.

- Keine Applikation kann (selbst mit Superuser-Rechten) auf den Kernelspace zugreifen !